

✓ 06 — Baseline decoding models

I train classical decoders before using advanced models. This notebook treats real movie-frame visual statistics as decoding targets, handles imbalanced and ambiguous dominant-orientation labels explicitly, and reports null baselines. I treat continuous visual-feature decoding as the primary analysis; coarse/confident orientation decoding is secondary or exploratory.

```
from pathlib import Path
import sys

ROOT = Path.cwd()
if ROOT.name == "notebooks":
    ROOT = ROOT.parent
if str(ROOT / "src") not in sys.path:
    sys.path.insert(0, str(ROOT / "src"))

from v1_manifold.config import load_config, get_paths, set_global_seed
from v1_manifold.visualization import set_publication_style, save_figure
from v1_manifold.utils import save_table

cfg = load_config(ROOT / "configs" / "default.yaml")
cfg["paths"]["root"] = str(ROOT)
paths = get_paths(cfg)
set_global_seed(cfg["project"]["random_seed"])
set_publication_style()
print(f"Project root: {ROOT}")
```

Project root: c:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-na

```
import numpy as np
import pandas as pd
from v1_manifold.models import evaluate_decoders, evaluate_regressors, save_f:
from v1_manifold.preprocessing import load_trial_tensor_h5
from v1_manifold.features import (
    assert_real_stimulus_features,
    make_single_trial_design_matrix,
    add_coarse_orientation_target,
    add_confident_orientation_target,
    summarize_class_balance,
    safe_classification_target,
    balanced_confident_orientation_subset,
)
from v1_manifold.visualization import plot_metric_bar, plot_confusion_matrix
from sklearn.metrics import confusion_matrix
```

```

import matplotlib.pyplot as plt

h5_files = sorted(paths.interim_dir.glob("session*_tensor.h5"))
emb_files = sorted(paths.processed_dir.glob("session*_embeddings.npz"))
if not h5_files:
    raise FileNotFoundError("No trial tensor exists yet. I need to run notebook 04")
if not emb_files:
    raise FileNotFoundError("No embeddings exist yet. I need to run notebook 04")

tensor_path = h5_files[0]
emb_path = emb_files[0]
session_id = emb_path.name.split("_")[1]

feature_candidates = sorted(paths.processed_dir.glob(f"session_{session_id}*_features.csv"))
if not feature_candidates:
    raise FileNotFoundError("Frame features not found. I need to run notebook 04")
features = pd.read_csv(feature_candidates[0])
assert_real_stimulus_features(features)
if "dominant_orientation_coarse" not in features.columns:
    features = add_coarse_orientation_target(
        features,
        top_k=cfg["features"].get("coarse_orientation_top_k", 2),
    )
if "dominant_orientation_confident" not in features.columns:
    features = add_confident_orientation_target(
        features,
        confidence_quantile=cfg["features"].get("orientation_confidence_quantile", 0.5),
        ambiguous_label=cfg["features"].get("orientation_ambiguous_label", "arbitrary")
    )

# I evaluate two complementary decoding settings:
# 1) frame-level decoding from latent embeddings, and
# 2) repeat-wise held-out single-trial decoding from raw neural activity.
emb = np.load(emb_path)
tensor, R = load_trial_tensor_h5(tensor_path)
X_trial, repeat_group, frame_idx = make_single_trial_design_matrix(tensor)

# The embeddings file can also contain metadata arrays such as PCA variance scores
# I keep only arrays shaped as frame-level embeddings: [n_movie_frames, n_latent_dimensions]
embedding_summary = describe_embedding_file(emb, n_samples=len(features))
save_table(embedding_summary, paths.tables_dir / f"06_embedding_file_summary.csv")
display(embedding_summary)

embedding_names = valid_embedding_names(emb, n_samples=len(features))
if not embedding_names:
    raise ValueError(
        "No valid 2D frame-level embeddings were found in the embeddings file. "
        "I need notebook 05 to save arrays such as pca, umap, isomap, or cebra."
    )
print("Valid frame-level embeddings used for latent decoding:", embedding_names)

```

```
# I make the label imbalance explicit before training any classifier.
class_balance_tables = []
for target in ["dominant_orientation_bin", "dominant_orientation_coarse", "dor
    if target in features.columns:
        tab = summarize_class_balance(features[target], target_name=target)
        tab.insert(0, "target", target)
        class_balance_tables.append(tab)
class_balance = pd.concat(class_balance_tables, ignore_index=True)
save_table(class_balance, paths.tables_dir / "06_classification_target_balance
display(class_balance)

classification_targets = {
    "dominant_orientation_coarse_secondary": features["dominant_orientation_co
    "dominant_orientation_confident_exploratory": features["dominant_orientat
    "spatial_frequency_bin_secondary": features["spatial_frequency_bin"].to_nu
    # Fine 8-bin orientation is kept as exploratory because it is often highly
    "dominant_orientation_bin_fine_exploratory": features["dominant_orientatio
}

# I skip classification targets that do not have enough support for cross-val
classification_targets = {
    name: y for name, y in classification_targets.items()
    if safe_classification_target(y, min_classes=2, min_count_per_class=2)
}

# I construct a stricter balanced confident-orientation subset by excluding ar
# frames and downsampling the majority confident class. This is intentionally
# and exploratory, but it is fairer than decoding the full imbalanced labels.
balanced_confident = balanced_confident_orientation_subset(
    features,
    random_state=cfg["project"]["random_seed"],
)
if not balanced_confident.empty:
    save_table(
        summarize_class_balance(
            balanced_confident["dominant_orientation_confident"],
            target_name="dominant_orientation_confident_balanced",
        ),
        paths.tables_dir / "06_balanced_confident_orientation_subset_balance.c
    )

regression_targets = {
    "rms_contrast": features["rms_contrast"].to_numpy(),
    "spatial_frequency_centroid": features["spatial_frequency_centroid"].to_nu
    "orientation_selectivity": features["orientation_selectivity"].to_numpy(),
    "luminance_std": features["luminance_std"].to_numpy(),
    "total_spectral_power": features["total_spectral_power"].to_numpy(),
}

rows = []
```

```

all_predictions = {}
include_nulls = cfg["evaluation"].get("include_null_baselines", True)

# Frame-level latent decoding. This asks whether learned manifold coordinates
# I interpret this as exploratory because adjacent movie frames are autocorrelated.
for target_name, y_frame in classification_targets.items():
    for embedding_name in embedding_names:
        df, preds, fitted = evaluate_decoders(
            emb[embedding_name],
            y_frame,
            n_splits=cfg["evaluation"]["n_splits"],
            random_state=cfg["project"]["random_seed"],
            include_null_baselines=include_nulls,
        )
        df.insert(0, "target", target_name)
        df.insert(1, "analysis", "frame_level_latent_exploratory")
        df.insert(2, "embedding", embedding_name)
        rows.append(df)
        all_predictions[("frame_level_latent_exploratory", target_name, embedding_name)] = preds
        save_fitted_models(fitted, paths.models_dir, prefix=f"session_{session_id}_{target_name}_{embedding_name}")

# Repeat-wise single-trial decoding. This is the more neuroscience-faithful heuristic.
for target_name, y_frame in classification_targets.items():
    y_trial = y_frame[frame_idx]
    df, preds, fitted = evaluate_decoders(
        X_trial,
        y_trial,
        groups=repeat_group,
        n_splits=min(cfg["evaluation"]["n_splits"], len(np.unique(repeat_group))),
        random_state=cfg["project"]["random_seed"],
        include_null_baselines=include_nulls,
    )
    df.insert(0, "target", target_name)
    df.insert(1, "analysis", "single_trial_repeat_heldout")
    df.insert(2, "embedding", "raw_cells")
    rows.append(df)
    all_predictions[("single_trial_repeat_heldout", target_name, "raw_cells")] = preds
    save_fitted_models(fitted, paths.models_dir, prefix=f"session_{session_id}_{target_name}_raw_cells")

# Balanced confident-only orientation decoding: secondary/exploratory analysis
if not balanced_confident.empty and safe_classification_target(
    balanced_confident["dominant_orientation_confident"],
    min_classes=2,
    min_count_per_class=2,
):
    target_name = "dominant_orientation_confident_balanced_binary_exploratory"
    frame_subset = balanced_confident["movie_frame"].astype(int).to_numpy()
    y_balanced = balanced_confident["dominant_orientation_confident"].to_numpy()

    for embedding_name in embedding_names:
        df, preds, fitted = evaluate_decoders(

```

```

df, preds, fitted = evaluate_decoders(
    emb[embedding_name][frame_subset],
    y_balanced,
    n_splits=min(cfg["evaluation"]["n_splits"], pd.Series(y_balanced).
        random_state=cfg["project"]["random_seed"],
        include_null_baselines=include_nulls,
    )
df.insert(0, "target", target_name)
df.insert(1, "analysis", "balanced_confident_frame_level_exploratory")
df.insert(2, "embedding", embedding_name)
rows.append(df)
all_predictions[("balanced_confident_frame_level_exploratory", target_name)] = preds
save_fitted_models(fitted, paths.models_dir, prefix=f"session_{session_id}")

frame_to_label = dict(zip(frame_subset, y_balanced))
trial_mask = np.isin(frame_idx, frame_subset)
y_trial_balanced = np.array([frame_to_label[int(i)] for i in frame_idx[trial_mask]])
df, preds, fitted = evaluate_decoders(
    X_trial[trial_mask],
    y_trial_balanced,
    groups=repeat_group[trial_mask],
    n_splits=min(cfg["evaluation"]["n_splits"], len(np.unique(repeat_group[trial_mask])),
        random_state=cfg["project"]["random_seed"],
        include_null_baselines=include_nulls,
    )
df.insert(0, "target", target_name)
df.insert(1, "analysis", "balanced_confident_single_trial_repeat_heldout")
df.insert(2, "embedding", "raw_cells")
rows.append(df)
all_predictions[("balanced_confident_single_trial_repeat_heldout", target_name)] = preds
save_fitted_models(fitted, paths.models_dir, prefix=f"session_{session_id}")

if not rows:
    raise RuntimeError("No classification targets had enough support for cross-validation")

benchmark = pd.concat(rows, ignore_index=True)
benchmark.insert(0, "session_id", session_id)
save_table(benchmark, paths.tables_dir / "06_baseline_decoding_benchmark.csv")
display(benchmark.sort_values(["target", "balanced_accuracy"], ascending=[True, False]))

# Continuous visual-feature decoding from held-out repeats. This is the primary analysis.
regression_rows = []
for target_name, y_frame in regression_targets.items():
    y_trial = y_frame[frame_idx]
    df, preds, fitted = evaluate_regressors(
        X_trial,
        y_trial,
        groups=repeat_group,
        n_splits=min(cfg["evaluation"]["n_splits"], len(np.unique(repeat_group))),
        random_state=cfg["project"]["random_seed"],
    )

```

```
df.insert(0, "session_id", session_id)
df.insert(1, "target", target_name)
df.insert(2, "analysis", "single_trial_repeat_heldout")
df.insert(3, "embedding", "raw_cells")
regression_rows.append(df)
save_fitted_models(fitted, paths.models_dir, prefix=f"session_{session_id}")

regression_benchmark = pd.concat(regression_rows, ignore_index=True)
save_table(regression_benchmark, paths.tables_dir / "06_continuous_feature_decoding")
display(regression_benchmark.sort_values("r2", ascending=False).head(20))
```

	array_name	shape	ndim	is_valid_frame_embedding
0	pca	(900, 3)	2	True
1	pca_full	(900, 20)	2	True
2	umap	(900, 3)	2	True
3	isomap	(900, 3)	2	True
4	frame	(900,)	1	False
5	rms_contrast	(900,)	1	False
6	spatial_frequency_centroid	(900,)	1	False

Valid frame-level embeddings used for latent decoding: ['pca', 'pca_full', 'umap', 'isomap']

	target	dominant_orientation_bin	n_samples	fraction
0	dominant_orientation_bin	0.0	13	0.014444
1	dominant_orientation_bin	1.0	33	0.036667
2	dominant_orientation_bin	3.0	557	0.618889
3	dominant_orientation_bin	4.0	287	0.318889
4	dominant_orientation_bin	6.0	8	0.008889
5	dominant_orientation_bin	7.0	2	0.002222
6	dominant_orientation_coarse	NaN	557	0.618889
7	dominant_orientation_coarse	NaN	287	0.318889
8	dominant_orientation_coarse	NaN	56	0.062222
9	dominant_orientation_confident	NaN	450	0.500000
10	dominant_orientation_confident	NaN	392	0.435556
11	dominant_orientation_confident	NaN	58	0.064444
12	spatial_frequency_bin	NaN	900	1.000000

session_id	target	analysis
500055014	dominant_orientation_bin	single_trial_repeat_heldout

72	500855614	dominant_orientation_bin_fine_exploratory	frame_level_latent_explorato
73	500855614	dominant_orientation_bin_fine_exploratory	frame_level_latent_explorato
80	500855614	dominant_orientation_bin_fine_exploratory	frame_level_latent_explorato
74	500855614	dominant_orientation_bin_fine_exploratory	frame_level_latent_explorato
75	500855614	dominant_orientation_bin_fine_exploratory	frame_level_latent_explorato
81	500855614	dominant_orientation_bin_fine_exploratory	frame_level_latent_explorato
76	500855614	dominant_orientation_bin_fine_exploratory	frame_level_latent_explorato
82	500855614	dominant_orientation_bin_fine_exploratory	frame_level_latent_explorato
112	500855614	dominant_orientation_bin_fine_exploratory	single_trial_repeat_heldo
88	500855614	dominant_orientation_bin_fine_exploratory	frame_level_latent_explorato
83	500855614	dominant_orientation_bin_fine_exploratory	frame_level_latent_explorato
89	500855614	dominant_orientation_bin_fine_exploratory	frame_level_latent_explorato
84	500855614	dominant_orientation_bin_fine_exploratory	frame_level_latent_explorato
90	500855614	dominant_orientation_bin_fine_exploratory	frame_level_latent_explorato
113	500855614	dominant_orientation_bin_fine_exploratory	single_trial_repeat_heldo
64	500855614	dominant_orientation_bin_fine_exploratory	frame_level_latent_explorato
65	500855614	dominant_orientation_bin_fine_exploratory	frame_level_latent_explorato
114	500855614	dominant_orientation_bin_fine_exploratory	single_trial_repeat_heldo
66	500855614	dominant_orientation_bin_fine_exploratory	frame_level_latent_explorato
91	500855614	dominant_orientation_bin_fine_exploratory	frame_level_latent_explorato

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

	session_id	target	analysis	embedding	
16	500855614	total_spectral_power	single_trial_repeat_heldout	raw_cells	15
12	500855614	luminance_std	single_trial_repeat_heldout	raw_cells	
0	500855614	rms_contrast	single_trial_repeat_heldout	raw_cells	
1	500855614	rms_contrast	single_trial_repeat_heldout	raw_cells	
13	500855614	luminance_std	single_trial_repeat_heldout	raw_cells	
17	500855614	total_spectral_power	single_trial_repeat_heldout	raw_cells	17
8	500855614	orientation_selectivity	single_trial_repeat_heldout	raw_cells	
9	500855614	orientation_selectivity	single_trial_repeat_heldout	raw_cells	
4	500855614	spatial_frequency_centroid	single_trial_repeat_heldout	raw_cells	
5	500855614	spatial_frequency_centroid	single_trial_repeat_heldout	raw_cells	
2	500855614	rms_contrast	single_trial_repeat_heldout	raw_cells	
14	500855614	luminance_std	single_trial_repeat_heldout	raw_cells	

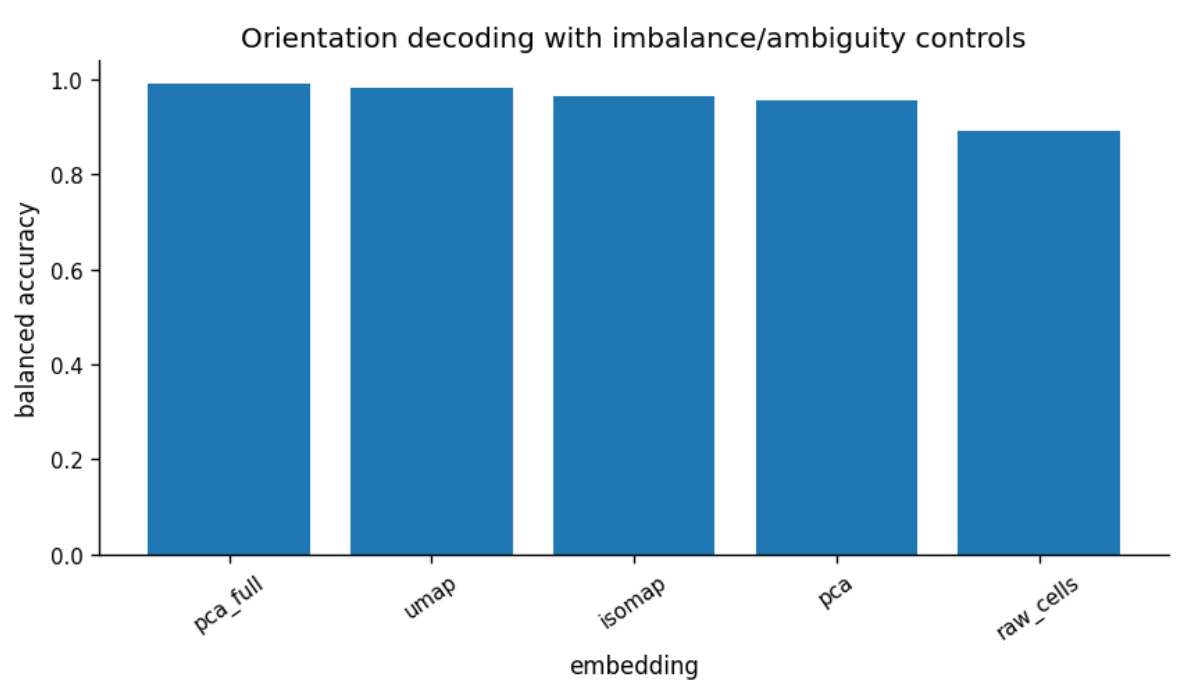
18	500855614	total_spectral_power	single_trial_repeat_heldout	raw_cells	200
10	500855614	orientation_selectivity	single_trial_repeat_heldout	raw_cells	
15	500855614	luminance_std	single_trial_repeat_heldout	raw_cells	
3	500855614	rms_contrast	single_trial_repeat_heldout	raw_cells	
19	500855614	total_spectral_power	single_trial_repeat_heldout	raw_cells	200
11	500855614	orientation_selectivity	single_trial_repeat_heldout	raw_cells	
6	500855614	spatial_frequency_centroid	single_trial_repeat_heldout	raw_cells	
7	500855614	spatial_frequency_centroid	single_trial_repeat_heldout	raw_cells	

```

plot_df = benchmark[benchmark["target"].str.contains("dominant_orientation",

fig = plot_metric_bar(
    plot_df,
    x="embedding",
    y="balanced_accuracy",
    title="Orientation decoding with imbalance/ambiguity controls",
)
save_figure(fig, paths.figures_dir / "06_coarse_orientation_decoding_compari
plt.show()

```




```

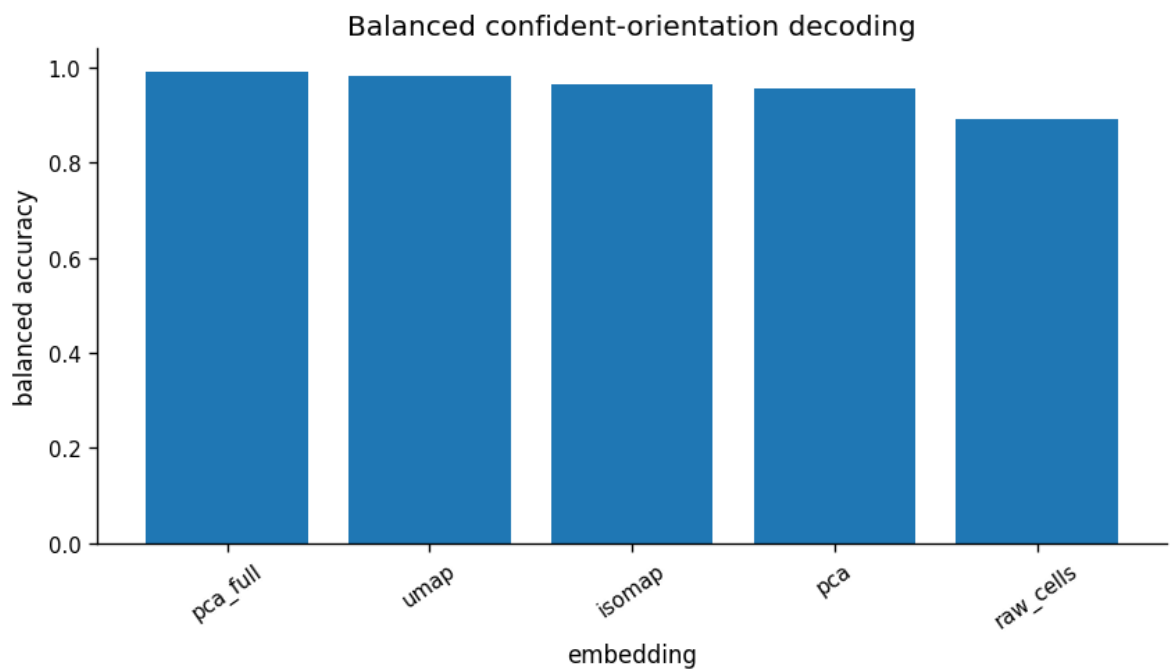
plot_df = benchmark[
    (~benchmark["is_null_baseline"].astype(bool))
    & (benchmark["target"] == "dominant_orientation_confident_balanced_binar
].copy()

fig = plot_metric_bar(
    plot_df,
    x="embedding",
    y="balanced_accuracy",
    title="Balanced confident-orientation decoding",
)

save_figure(
    fig,
    paths.figures_dir / "06_balanced_confident_orientation_decoding_comparis
)

plt.show()

```



```

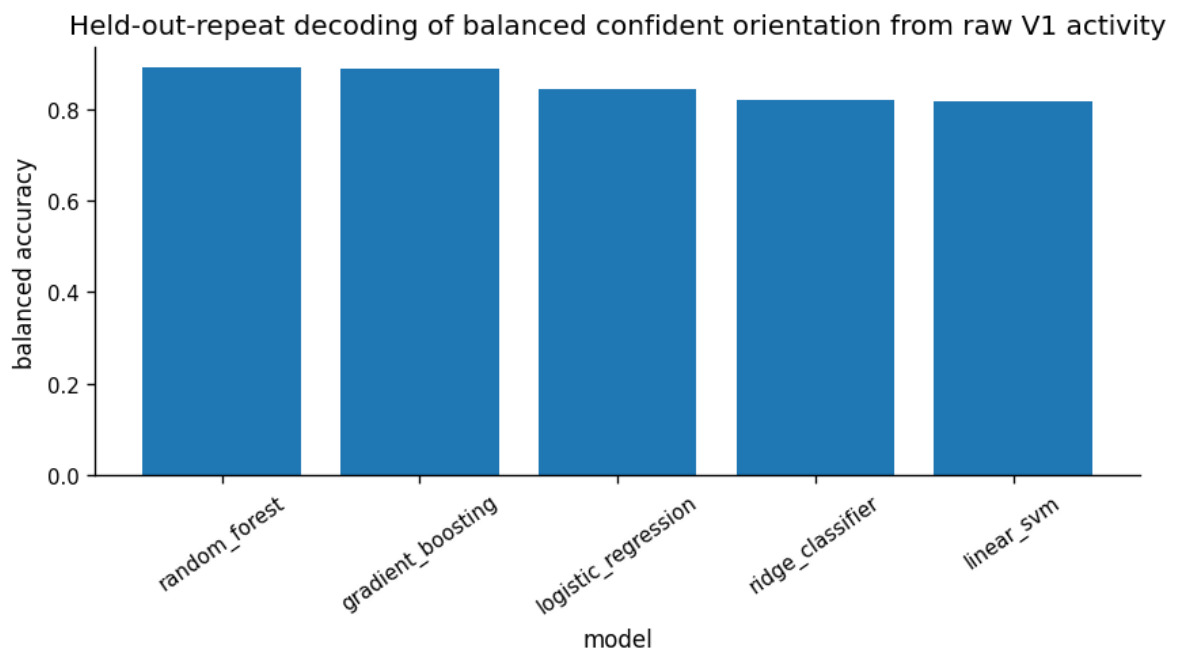
plot_df = benchmark[
    (~benchmark["is_null_baseline"].astype(bool))
    & (benchmark["target"] == "dominant_orientation_confident_balanced_binar
    & (benchmark["analysis"] == "balanced_confident_single_trial_repeat_held
].copy()

fig = plot_metric_bar(
    plot_df,
    x="model",
    y="balanced_accuracy",
    title="Held-out-repeat decoding of balanced confident orientation from r
)

save_figure(
    fig,
    paths.figures_dir / "06_raw_cells_balanced_confident_orientation_repeat_
)

plt.show()

```



```

# This cell reconstructs the correct y_true for ordinary targets and for the
# balanced confident-orientation subset.

non_null = benchmark[~benchmark["is_null_baseline"].astype(bool)].copy()

if non_null.empty:
    raise RuntimeError("No non-null decoder results are available for confus

best = non_null.sort_values("balanced_accuracy", ascending=False).iloc[0]

key = (
    best["analysis"],
    best["target"],
    best["embedding"],
)

if key not in all_predictions:
    raise KeyError(
        f"The prediction key {key} was not found in all_predictions. "
        "This usually means the notebook kernel was restarted after the expe
    )

y_pred = all_predictions[key][best["model"]]

target = best["target"]
analysis = best["analysis"]

if target == "dominant_orientation_confident_balanced_binary_exploratory":
    frame_subset = balanced_confident["movie_frame"].astype(int).to_numpy()
    y_balanced = balanced_confident["dominant_orientation_confident"].to_num

    if analysis == "balanced_confident_frame_level_exploratory":
        y_true = y_balanced

    elif analysis == "balanced_confident_single_trial_repeat_heldout":
        frame_to_label = dict(zip(frame_subset, y_balanced))
        trial_mask = np.isin(frame_idx, frame_subset)
        y_true = np.array([frame_to_label[int(i)] for i in frame_idx[trial_m

    else:
        raise ValueError(
            f"Unexpected analysis for balanced confident target: {analysis}"
        )

else:
    if target not in classification_targets:
        raise KeyError(
            f"{target} was not found in classification_targets. "
            "This target may be a derived subset and needs explicit handling
        )

```

```

        if analysis == "single_trial_repeat_heldout":
            y_true = classification_targets[target][frame_idx]
        else:
            y_true = classification_targets[target]

# Safety check before plotting.
if len(y_true) != len(y_pred):
    raise ValueError(
        f"Confusion-matrix length mismatch: y_true has {len(y_true)} samples"
        f"but y_pred has {len(y_pred)} predictions."
    )

labels_sorted = sorted(np.unique(np.concatenate([np.asarray(y_true), np.asar

cm = confusion_matrix(
    y_true,
    y_pred,
    labels=labels_sorted,
)

fig = plot_confusion_matrix(
    cm,
    labels=labels_sorted,
    title=(
        f"Best non-null decoder\n"
        f"{best['target']} / {best['analysis']} / {best['embedding']} / {bes
    ),
)

save_figure(
    fig,
    paths.figures_dir / "06_best_non_null_real_feature_confusion_matrix.png"
)

confusion_summary = pd.DataFrame({
    "field": [
        "session_id",
        "target",
        "analysis",
        "embedding",
        "model",
        "n_samples",
        "accuracy",
        "balanced_accuracy",
        "macro_f1",
    ],
    "value": [
        best["session_id"],
        best["target"],
        best["analysis"],

```

```

        best["analysis"],
        best["embedding"],
        best["model"],
        len(y_true),
        best["accuracy"],
        best["balanced_accuracy"],
        best["macro_f1"],
    ],
})

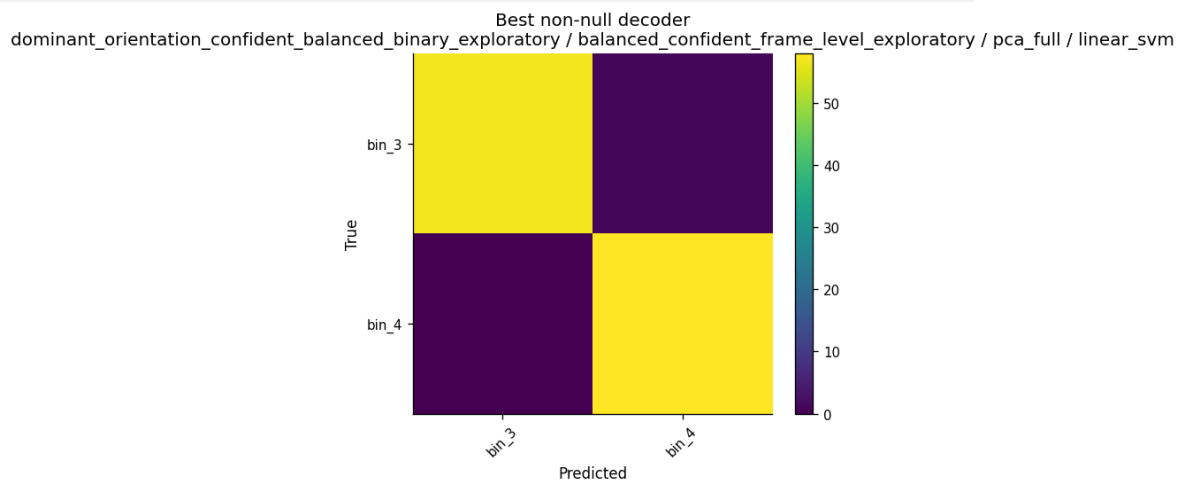
save_table(
    confusion_summary,
    paths.tables_dir / "06_best_non_null_confusion_matrix_summary.csv",
)

display(confusion_summary)

plt.show()

```

	field	value
0	session_id	500855614
1	target	dominant_orientation_confident_balanced_binary...
2	analysis	balanced_confident_frame_level_exploratory
3	embedding	pca_full
4	model	linear_svm
5	n_samples	116
6	accuracy	0.991379
7	balanced_accuracy	0.991379
8	macro_f1	0.991379

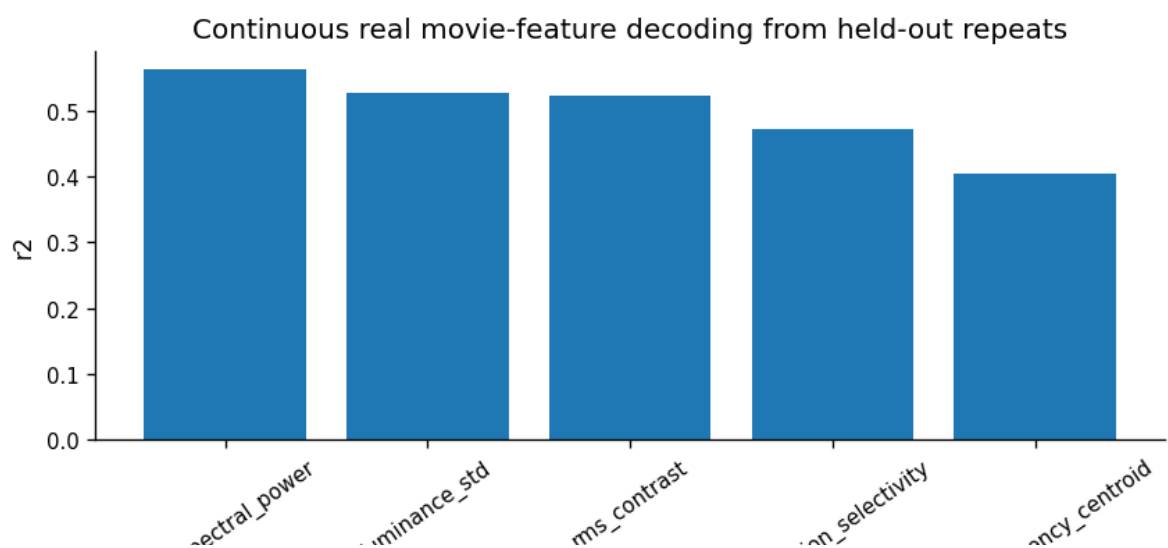


```

fig = plot_metric_bar(
    regression_benchmark,
    x="target",
    y="r2",
    title="Continuous real movie-feature decoding from held-out repeats",
)
save_figure(fig, paths.figures_dir / "06_continuous_feature_decoding_r2.png")
plt.show()

primary_regression_summary = (
    regression_benchmark
    .sort_values("r2", ascending=False)
    .groupby("target", as_index=False)
    .first()
)
save_table(
    primary_regression_summary,
    paths.tables_dir / "06_primary_continuous_decoding_summary.csv",
)
display(primary_regression_summary)

```



target				
	target	session_id	analysis	embedding
0	luminance_std	500855614	single_trial_repeat_heldout	raw_cells
1	orientation_selectivity	500855614	single_trial_repeat_heldout	raw_cells
2	rms_contrast	500855614	single_trial_repeat_heldout	raw_cells
3	spatial_frequency_centroid	500855614	single_trial_repeat_heldout	raw_cells
4	total_spectral_power	500855614	single_trial_repeat_heldout	raw_cells